AMITY INSTITUTE OF INFORMATION TECHNOLOGY

Bachelor of Computer Applications

Laboratory E-Record

Introduction to AI and AL

CSE2052

3rd Semester - 2025-26

SUBMITTED TO:

Dr. MADHU BHAN
Associate Professor

SUBMITTED BY:

Student Name : Thrisha M
SEN : A86904824001

Amity University Bengaluru

NH-207, opposite to the office of Deputy Commissioner, Devanahalli,
Doddaballapura, Bangalore – 562110

AMITY INSTITUTE OF INFORMATION TECHNOLOGY

INDEX

S.No.	Exercise No	Date
1	LAB EXERCISE-01	24/09/25
2	LAB EXERCISE-02	24/09/25
3	LAB EXERCISE-03	08/10/25
4	LAB EXERCISE-04	08/10/25
5	LAB EXERCISE-05	15/10/25
6	LAB EXERCISE-06	29/10/25
7	LAB EXERCISE-07	12/11/25
8	LAB EXERCISE-08	19/11/25
9	LAB EXERCISE-09	19/11/25
10	LAB EXERCISE-10	26/11/25
11	LAB EXERCISE-11	26/11/25
12	LAB EXERCISE-12	03/12/25
13	LAB EXERCISE-13	10/12/25
14	LAB EXERCISE-14	10/12/25
15	LAB EXERCISE-15	24/12/25
16	LAB EXERCISE-16	31/12/25
17	LAB EXERCISE-17	07/01/26
18	LAB EXERCISE-18	21/01/26
19	LAB EXERCISE-19	21/01/26
20	LAB EXERCISE-20	28/01/26

CSE2052 - Lab Exercise 1

Program 1 : Simple Rule-Based Agent.

1. Write a Python program where the agent decides an action based on conditions. Example: If it's "sunny" -> print "Wear sunglasses", If it's "rainy" -> print "Carry umbrella"

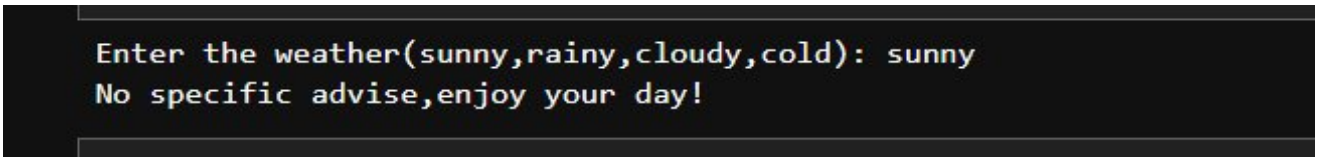
Code :

```
weather=input("Enter the weather(sunny,rainy,cloudy,cold):").lower()
if weather=="Sunny":
   print ("Wear Sunglasses")
elif weather=="Rainy":
   print ("Carry an Umbrella")
elif weather=="Cloudy":
   print ("It might rain,better take a light jacket")
elif weather=="Cold":
   print ("Wear a warm coat")
else:
   print("No specific advise,enjoy your day!")
```

Output :

```
Enter the weather(sunny,rainy,cloudy,cold): rainy
No specific advise,enjoy your day!
```

Snapshot :



CSE2052 - Lab Exercise 2

Program 2 : Simple Rule-Based Agent.

2. Write a Python program where the agent decides an action based on Temperature. Example: If temp > 30 -> print "Drink plenty of water": else If it's temp > 20 -> print "Weather is pleasant": else if temp > 10 -> print "Wear a jacket"

Code :

```
temperature=input("Enter the temperature(10,20,30,40):").lower()
if temperature =="10":
   print("wear jacket")
elif temperature =="20":
   print("weather is pleasent")
elif temperature =="30":
   print("drink plenty of water")
```

```
elif temperature =="40":
   print("too high temperature")
else:
   print("no specific advise,enjoy your day!")
```

Output :

Enter the temperature(10,20,30,40): 20
weather is pleasent

Snapshot :

```
Enter the temperature(10,20,30,40): 20
weather is pleasent
```

CSE2052 - Lab Exercise 3

Program 3 : Problem Representation.

3. Write a Python program : 8-Puzzle 8-Puzzle State Pepresentation (8-puzzle new). Display as 3 * 3 Matrix. Compare initial state and goal state and if equal display “The two states are same” : else display “The two states are different.

Code :

```
initial_state=[1,2,3,4,5,6,7,8,-1]
goal_state=[1,2,3,4,5,6,7,8,-1]
print("Initial state :")
for i in range(0,9,3):
   print(initial_state[i:i+3])
print("\nGoal state :")
for i in range(0,9,3):
   print(goal_state[i:i+3])
if initial_state == goal_state:
   print("\nThe two states are same")
else:
   print("\nThe two states are different")
```

Output :

Initial state :
[1, 2, 3]
[4, 5, 6]
[7, 8, -1]

Goal state :
[1, 2, 3]
[4, 5, 6]
[7, 8, -1]

The two states are same

Snapshot :

```
Initial state :
[1, 2, 3]
[4, 5, 6]
[7, 8, -1]

Goal state :
[1, 2, 3]
[4, 5, 6]
[7, 8, -1]

The two states are same
```

CSE2052 - Lab Exercise 4

Program 4 : Simple Rule-Based Agent

4. Write a Python program to simulate a simple Vacuum Cleaner Agent that cleans two locations (A and B). The agent should check if the current location is dirty — if it is, clean it; otherwise, move to the next location. Finally, display the initial and final states of the environment. (vacuum cleaner)

Code :

```
def vacuum_cleaner_agent():
    environment = {
        "A": input("Enter status of location A (clean/dirty): ").lower(),
        "B": input("Enter status of location B (clean/dirty): ").lower()
    }
    print("Initial Environment State:", environment)
    current_location = "A"
    for location in ["A", "B"]:
        current_location = location
        print("Vacuum is now at location {A}")
        if environment[location] == "dirty":
            print("Location {B} is dirty. Cleaning now.")
            environment[location] = "clean"
            print("Location {B} cleaned.")
        else:
            print("Location {B} is already clean. Moving to next location.")
    print("Final Environment State:", environment)
    print("All locations are clean. Task complete")
```

```
vacuum_cleaner_agent()
```

Output :

```
Enter status of location A (clean/dirty):  clean
Enter status of location B (clean/dirty):  dirty
Initial Environment State: {'A': 'clean', 'B': 'dirty'}
Vacuum is now at location {A}
Location {B} is already clean. Moving to next location.
Vacuum is now at location {A}
Location {B} is dirty. Cleaning now.
Location {B} cleaned.
Final Environment State: {'A': 'clean', 'B': 'clean'}
All locations are clean. Task complete
```

Snapshot :

```
Enter status of location A (clean/dirty):  clean
Enter status of location B (clean/dirty):  clean
Initial Environment State: {'A': 'clean', 'B': 'clean'}
Vacuum is now at location {A}
Location {B} is already clean. Moving to next location.
Vacuum is now at location {A}
Location {B} is already clean. Moving to next location.
Final Environment State: {'A': 'clean', 'B': 'clean'}
All locations are clean. Task complete
```

CSE2052 - Lab Exercise 5

Program 5 : Goal Based Agent

5. Write a Python program to simulate an Autonomous Car Agent that can make driving decisions based on environmental conditions such as traffic light color, obstacle detection, and road condition.
   The car should decide whether to move, stop, or slow down depending on the situation. *(car simulation)*

Code :

```
def autonomous_car_agent(traffic_light, obstacle, road_condition):
    print("Traffic Light:", traffic_light)
    print("Obstacle Detected:", obstacle)
    print("Road Condition:", road_condition)
    if traffic_light.lower() == "red":
        action = "Stop the car"
    elif traffic_light.lower() == "yellow":
        action = "Slow down and prepare to stop"
    elif obstacle.lower() == "yes":
```

```
        action = "Stop — obstacle ahead"
    elif road_condition.lower() == "slippery":
        action = "Drive slowly and carefully"
    elif traffic_light.lower() == "green" and obstacle.lower() == "no":
        action = "Move forward safely"
    else:
        action = "Stay alert and maintain current state"
    print("Decision:", action)
print("=== Scenario 1 ===")
autonomous_car_agent("Red", "No", "Clear")
print("\n=== Scenario 2 ===")
autonomous_car_agent("Green", "Yes", "Clear")
print("\n=== Scenario 3 ===")
autonomous_car_agent("Green", "No", "Slippery")
```

Output :

```
=== Scenario 1 ===
Traffic Light: Red
Obstacle Detected: No
Road Condition: Clear
Decision: Stop the car

=== Scenario 2 ===
Traffic Light: Green
Obstacle Detected: Yes
Road Condition: Clear
Decision: Stop — obstacle ahead

=== Scenario 3 ===
Traffic Light: Green
Obstacle Detected: No
Road Condition: Slippery
Decision: Drive slowly and carefully
```

Snapshot :

```
=== Scenario 1 ===
Traffic Light: Red
Obstacle Detected: No
Road Condition: Clear
Decision: Stop the car

=== Scenario 2 ===
Traffic Light: Green
Obstacle Detected: Yes
Road Condition: Clear
Decision: Stop — obstacle ahead

=== Scenario 3 ===
Traffic Light: Green
Obstacle Detected: No
Road Condition: Slippery
Decision: Drive slowly and carefully
```

CSE2052 - Lab Exercise 6

Program 6 : Breadth First Search (BFS)

6. Write a Python program that accepts an adjacency list of a graph as input and performs **Breadth First Search (BFS)** starting from a given source node. The program should display the order in which the nodes are visited.

Code :

```
n = int(input("Enter the number of nodes: "))
nodes = []
print("Enter the node names:")
for i in range(n):
    name = input(f"Node {i+1} name: ")
    nodes.append(name)
graph = {}
print("\nEnter the adjacent nodes for each node (separated by space or comma):")
for node in nodes:
    raw = input(f"Neighbors of {node}: ")
    raw = raw.replace(",", " ")
    neighbors = raw.split()
    graph[node] = neighbors
print("\nAdjacency List Representation:")
for node in graph:
    print(node, "->", ", ".join(graph[node]))
start = input("\nEnter the starting node for BFS: ")
if start not in graph:
    print("\nError: Starting node not found in graph!")
    exit()
visited = []
queue = []
```

```
visited.append(start)
queue.append(start)
print("\nBFS Traversal Order:", end=" ")
while queue:
    node = queue.pop(0)
    print(node, end=" ")
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.append(neighbor)
            queue.append(neighbor)
print()
```

Output :

```
Enter the number of nodes:  6
Enter the node names:
Node 1 name:  A
Node 2 name:  B
Node 3 name:  C
Node 4 name:  D
Node 5 name:  E
Node 6 name:  F

Enter the adjacent nodes for each node (separated by space or comma):
Neighbors of A:  B,C
Neighbors of B:  A,D,E
Neighbors of C:  A,F
Neighbors of D:  B
Neighbors of E:  B
Neighbors of F:  C

Adjacency List Representation:
A -> B, C
B -> A, D, E
C -> A, F
D -> B
E -> B
F -> C

Enter the starting node for BFS:  A

BFS Traversal Order: A B C D E F
```

Snapshot :

```
Enter the number of nodes:  6
Enter the node names:
Node 1 name:  A
Node 2 name:  B
Node 3 name:  C
Node 4 name:  D
Node 5 name:  E
Node 6 name:  F

Enter the adjacent nodes for each node (separated by space or comma):
Neighbors of A:  B C
Neighbors of B:  A D E
Neighbors of C:  A F
Neighbors of D:  B
Neighbors of E:  B
Neighbors of F:  C

Adjacency List Representation:
A -> B, C
B -> A, D, E
C -> A, F
D -> B
E -> B
F -> C

Enter the starting node for BFS:  A

BFS Traversal Order: A B C D E F
```

CSE2052 - Lab Exercise 7

Program 7 : Depth First Search (DFS)

7. Write a Python program that accepts an adjacency list of a graph as input and performs **Depth First Search (DFS)** starting from a given source node. The program should display the order in which the nodes are visited.

Code :

```
n = int(input("Enter the number of nodes: "))
nodes = []
print("Enter node names: ")
for i in range(n):
   name = input("Node {} name:" . format(i + 1))
   nodes.append(name)
graph = {}
print("\n Enter the adjacent nodes for each node(separated by space).if none, press Enter:")
for node in nodes:
   neighbors = input("Neighbors of {}:".format(node)).split()
   graph[node]=neighbors
print("\n Adjacency List Representation:")
```

```
for node in graph:
    print(node,"->",",".join(graph[node]))
start = input("\n Enter starting node for DFS:")
visited = []
stack = []
visited.append(start)
stack.append(start)
print("\n DFS Traversal Order:",end = " ")
while stack:
    node = stack.pop()
    print(node,end = " ")
    for neighbor in reversed(graph[node]):
        if neighbor not in visited:
            visited.append(neighbor)
            stack.append(neighbor)
```

Output :

```
Enter the number of nodes:  7
Enter node names:
Node 1 name: s
Node 2 name: a
Node 3 name: b
Node 4 name: c
Node 5 name: d
Node 6 name: e
Node 7 name: f

 Enter the adjacent nodes for each node(separated by space).if none, press Enter:
Neighbors of s: a b
Neighbors of a: s c d
Neighbors of b: s e
Neighbors of c: a f
Neighbors of d: a
Neighbors of e: b
Neighbors of f: c

 Adjacency List Representation:
s -> a,b
a -> s,c,d
b -> s,e
c -> a,f
d -> a
e -> b
f -> c
```

Enter starting node for DFS: s

DFS Traversal Order: s a c f d b e

Snapshot :

```
Enter the number of nodes:  7
Enter node names:
Node 1 name: s
Node 2 name: a
Node 3 name: b
Node 4 name: c
Node 5 name: d
Node 6 name: e
Node 7 name: f

 Enter the adjacent nodes for each node(separated by space).if none, press Enter:
Neighbors of s: a b
Neighbors of a: s c d
Neighbors of b: s e
Neighbors of c: a f
Neighbors of d: a
Neighbors of e: b
Neighbors of f: c

 Adjacency List Representation:
s -> a,b
a -> s,c,d
b -> s,e
c -> a,f
d -> a
e -> b
f -> c

 Enter starting node for DFS: s

 DFS Traversal Order: s a c f d b e
```

<u>CSE2052 - Lab Exercise 8</u>

<u>Program 8</u> : Propositional logic expressions.

8. Write a Python program to evaluate basic **propositional logic expressions**. The user should enter truth values (**True/False**) for propositions **P** and **Q**. The program should compute and display the results of the following logical operations:
    - P AND Q
    - P OR Q
    - NOT P
    - P → Q (Implication)

Code :

```
print("Enter the truth values of P AND Q (True/False):")
P = input("Enter P(True/False):")
Q = input("Enter Q(True/False):")
if P == "True":
   P = True
else:
   P = False
if Q == "True":
   Q = True
else:
   Q = False
and_result = P and Q
or_result = P or Q
not_result = not P
implication_result = (not P) or Q
biconditional_result = (P and Q) or ((not P) and (not Q))
print("\n --- Result --- ")
print("P AND Q    =",and_result)
print("P OR Q     =",or_result)
print("NOT P      =",not_result)
print("P->Q       =",implication_result)
print("P<->Q      =",biconditional_result)
```

Output :

```
Enter the truth values of P AND Q (True/False):
Enter P(True/False): True
Enter Q(True/False): False

 --- Result ---
P AND Q    = False
P OR Q     = True
NOT P      = False
P->Q       = False
P<->Q      = False
```

Snapshot :

```
Enter the truth values of P AND Q (True/False):
Enter P(True/False): True
Enter Q(True/False): True

 --- Result ---
P AND Q     = True
P OR Q      = True
NOT P       = False
P->Q        = True
P<->Q       = True
```

CSE2052 - Lab Exercise 9

Program 9 : Storing predicate facts using simple lists.

9. Write a Python program that stores a few predicate facts (like parent, brother, teacher) using simple lists.
   The user will enter a query such as: parent (John, Mary).
   The program should check whether this fact exists in the lists and print:
   Fact exists / Fact does NOT exist (Predicate)

Code :

```
parent_facts = [("John","Marry"), ("Ravi","Amit")]
brother_facts = [("Amit","Rohan"), ("Rohan","Amit")]
teacher_facts = [("Seema","Class5"), ("Arun","Class3")]
print("Enter a Query like : Parent(John, Mary)")
query = input("Query: ")
parts = query.split("(")
predicate = parts[0]
args = parts[1].replace(")","")
arg1, arg2 =args.split(",")
arg1 = arg1.strip()
arg2 = arg2.strip()
fact_exists = False
if predicate == "Parent":
    if (arg1, arg2) in parent_facts:
        fact_exists = True
elif predicate == "Brother":
    if (arg1, arg2) in brother_facts:
        fact_exists = True
elif predicate == "Teacher":
    if (arg1, arg2) in teacher_facts:
        fact_exists = True
```

```
if fact_exists:
   print("Fact exists in the knowledge base.")
else:
   print("Fact does NOT exist in the knowledge base. ")
```

Output :

Enter a Query like : Parent(John, Mary)
Query: Brother(Amit,Rohan)
Fact exists in the knowledge base.

Snapshot :

```
Enter a Query like : Parent(John, Mary)
Query:  Brother(Amit,Rohan)
Fact exists in the knowledge base.
```

CSE2052 - Lab Exercise 10

Program 10 : Mean, Median, Mode, and Standard Deviation.

10. Find Mean, Median, Mode, and Standard Deviation of a given dataset:
    speed = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78, 77, 85, 86]

Code :

```
import numpy
from scipy import stats
speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]
x = numpy.mean(speed)
y = numpy.median(speed)
z= stats.mode(speed)
p = numpy.std(speed)
print( "mean is",x)
print("median is",y)
print("mode is",z)
print("std is",p
```

Output :

mean is 89.76923076923077
median is 87.0
mode is ModeResult(mode=np.int64(86), count=np.int64(3))
std is 9.258292301032677

Snapshot :

```
mean is 89.76923076923077
median is 87.0
mode is ModeResult(mode=np.int64(86), count=np.int64(3))
std is 9.258292301032677
```

CSE2052 - Lab Exercise 11

Program 11 : Dataset and features.

11. Write a Python program to demonstrate the concept of a dataset and features. Create a small dataset, identify the feature and target values, and display them separately. *(features, labels)*

Code :

```
import numpy as np
X = np.array({1, 2, 3, 4})
Y = np.array({10,20,30,40})
print("Feature before reshape (1-D):")
print(X)
print("Shape:", X.shape)
X=X.reshape(-1, 1)
print("\nFeature after reshape (2-D):")
print(X)
print("Shape:", X.shape)
Y =Y.reshape(-1, 1)
print("\nlable after reshape (2-D): ")
print(Y)
print("Shape:", Y.shape)
```

Output :

```
Feature before reshape (1-D):
{1, 2, 3, 4}
Shape: ()

Feature after reshape (2-D):
[[{1, 2, 3, 4}]]
Shape: (1, 1)

lable after reshape (2-D):
[[{40, 10, 20, 30}]]
Shape: (1, 1)
```

Snapshot :

```
Feature before reshape (1-D):
{1, 2, 3, 4}
Shape: ()

Feature after reshape (2-D):
[[{1, 2, 3, 4}]]
Shape: (1, 1)

lable after reshape (2-D):
[[{40, 10, 20, 30}]]
Shape: (1, 1)
```

CSE2052 - Lab Exercise 12

Program 12 : Simple Imputer

12. Design a Python program to demonstrate the use of **Simple Imputer** class for filling the missing values.

Code :

```
from sklearn.impute import SimpleImputer
import numpy as np

data = np.array([
   [10, 2],
   [15, np.nan],
   [np.nan, 4],
   [20, 5]
])
print("Original Data: ")
print(data)
imputer = SimpleImputer(strategy='mean')
imputed_data = imputer.fit_transform(data)
print("\nData After Imputation:" )
print(imputed_data)
```

Output :

```
Original Data:
[[10.  2.]
 [15. nan]
 [nan  4.]
 [20.  5.]]

Data After Imputation:
[[10.         2.        ]
 [15.         3.66666667]
```

```
 [15.        4.       ]
 [20.        5.       ]]
```

Snapshot :

```
Original Data:
[[10.  2.]
 [15. nan]
 [nan  4.]
 [20.  5.]]

Data After Imputation:
[[10.          2.        ]
 [15.          3.66666667]
 [15.          4.        ]
 [20.          5.        ]]
```

Code :

```
from sklearn.impute import SimpleImputer
import numpy as np
data = np.array([
    [10, 2],
    [15, np.nan],
    [np.nan, 4],
    [20, 5]
])
print("Original Data: ")
print(data)
imputer = SimpleImputer(strategy='median')
imputed_data = imputer.fit_transform(data)
print("\nData After Imputation:" )
print(imputed_data)
```

Output :

```
Original Data:
[[10.  2.]
 [15. nan]
 [nan  4.]
 [20.  5.]]

Data After Imputation:
[[10.  2.]
 [15.  4.]
 [15.  4.]
 [20.  5.]]
```

Snapshot :

```
Original Data:
[[10.  2.]
 [15. nan]
 [nan  4.]
 [20.  5.]]

Data After Imputation:
[[10.  2.]
 [15.  4.]
 [15.  4.]
 [20.  5.]]
```

Code :

```
from sklearn.impute import SimpleImputer
from scipy import stats
import numpy as np
data = np.array([
    [10, 2],
    [15, np.nan],
    [np.nan, 4],
    [20, 5]
])
print("Original Data: ")
print(data)
imputer = SimpleImputer(strategy='most_frequent')
imputed_data = imputer.fit_transform(data)
print("\nData After Imputation:" )
print(imputed_data)
```

Output :

```
Original Data:
[[10.  2.]
 [15. nan]
 [nan  4.]
 [20.  5.]]

Data After Imputation:
[[10.  2.]
 [15.  2.]
 [10.  4.]
 [20.  5.]]
```

Snapshot :

```
Original Data:
[[10.  2.]
 [15. nan]
 [nan  4.]
 [20.  5.]]

Data After Imputation:
[[10.  2.]
 [15.  2.]
 [10.  4.]
 [20.  5.]]
```

CSE2052 - Lab Exercise 13

Program 13 : Z-score.

13.Write a Python program to find outliers using **Z-score**.

Code :

```
import numpy as np
from scipy import stats
data = [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]
z_scores = stats.zscore(data)
threshold = 3
outliers = []
for i in range(len(data)):
    if abs(z_scores[i]) > threshold:
        outliers.append(data[i])
print("Data:", data)
print("Z-scores:", z_scores)
print("Outliers:", outliers)
```

Output :

```
Data: [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]
Z-scores: [-0.59665155 -0.51762485 -0.43859816 -0.39908481 -0.35957146 -0.28054477
 -0.20151807 -0.12249138 -0.04346468  2.95954972]
Outliers: []
```

Snapshot :

```
Data: [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]
Z-scores: [-0.59665155 -0.51762485 -0.43859816 -0.39908481 -0.35957146 -0.28054477
 -0.20151807 -0.12249138 -0.04346468  2.95954972]
Outliers: []
```

CSE2052 - Lab Exercise 14

Program 14 : Training data and testing data.

14. Design a Python program to demonstrate how a dataset is divided into training data and testing data using the **train_test_split()** function from the **scikit-learn** library.

Code :

```
import numpy as np
from sklearn.model_selection import train_test_split
x = np.array([800, 1000, 1200, 1500, 1800]).reshape(-1, 1)
y = np.array([120, 150, 180, 210, 250])
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
print("x_train:", x_train)
print("y_train:", y_train)
print("x_test:", x_test)
print("y_test:", y_test)
```

Output :

```
x_train: [[1800]
 [1200]
 [ 800]
 [1500]]
y_train: [250 180 120 210]
x_test: [[1000]]
y_test: [150]
```

Snapshot :

```
x_train: [[1800]
 [1200]
 [ 800]
 [1500]]
y_train: [250 180 120 210]
x_test: [[1000]]
y_test: [150]
```

Code :

```
import numpy as np
from sklearn.model_selection import train_test_split
x = np.array([800, 1000, 1200, 1500, 1800]).reshape(-1, 1)
y = np.array([120, 150, 180, 210, 250])
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2)
print("x_train:", x_train)
print("y_train:", y_train)
print("x_test:", x_test)
print("y_test:", y_test)
```

Output :

x_train: [[1800]
 [1500]
 [1000]
 [1200]]
y_train: [250 210 150 180]
x_test: [[800]]
y_test: [120]

Snapshot :

```
x_train: [[1800]
 [1000]
 [1200]
 [1500]]
y_train: [250 150 180 210]
x_test: [[800]]
y_test: [120]
```

2052 - Lab Exercise 15

Program 15 : Min-Max Normalization

15. Given the following dataset representing marks scored by students: [10, 12, 14, 15, 16, 18, 20, 22, 24]. Write a Python program to perform **Min-Max Normalization** to scale the data between **0 and 1** *(min-max)*.

Code :

```
import numpy as np
data = np.array([10, 12, 14, 15, 16, 18, 20, 22, 24])
data_min = np.min(data)
data_max = np.max(data)
normalized_data = (data - data_min) / (data_max - data_min)
print("Original Data:", data)
print("Minimum Value:", data_min)
print("Maximum Value:", data_max)
print("Normalized Data:", normalized_data)
```

Output :

Original Data: [10 12 14 15 16 18 20 22 24]
Minimum Value: 10
Maximum Value: 24
Normalized Data: [0. 0.14285714 0.28571429 0.35714286 0.42857143 0.57142857
 0.71428571 0.85714286 1.

Snapshot :

```
Original Data: [10 12 14 15 16 18 20 22 24]
Minimum Value: 10
Maximum Value: 24
Normalized Data: [0.         0.14285714 0.28571429 0.35714286 0.42857143 0.57142857
 0.71428571 0.85714286 1.        ]
```

CSE2052 - Lab Exercise 16

Program 16 : Training data and testing data using scikit-learn to implement Simple Linear Regression.

16. Design a simple python program to implement Simple Linear Regression using Python and scikit-learn to predict house prices on house size.

Code :

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
x = np.array([800, 1000, 1200, 1500, 1800]).reshape(-1, 1)
y = np.array([120, 150, 180, 210, 250])
x_train,x_test, y_train, y_test = train_test_split(
   x, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
Predicted_Price = model.predict([[300]])
print("Test Data (x_test):", x_test)
print("Actual Price (y_test):", y_test)
print("Predicted Price (y_pred):", y_pred)
print("Slope (Coefficient):",model.coef_)
print("Intercept:",model.intercept_)
print("Price value for area 300sqt:",Predicted_Price)
```

Output :

Test Data (x_test): [[1000]]
Actual Price (y_test): [150]
Predicted Price (y_pred): [148.44748858]
Slope (Coefficient): [0.12785388]
Intercept: 20.59360730593616
Price value for area 300sqt: [58.94977169]

Snapshot :

```
Test Data (x_test): [[1000]]
Actual Price (y_test): [150]
Predicted Price (y_pred): [148.44748858]
Slope (Coefficient): [0.12785388]
Intercept: 20.593607305936075
Price value for area 300sqt: [58.94977169]
```

Code :

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
x = np.array([800, 1000, 1200, 1500, 1800]).reshape(-1, 1)
y = np.array([120, 150, 180, 210, 250])
x_train, x_test, y_train,y_test = train_test_split(
   x, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("test data (x_test):", x_test)
print("actual price (y_test):", y_test)
print("predicted price (y_pred):", y_pred)
```

Output :

test data (x_test): [[1000]]
actual price (y_test): [150]
predicted price (y_pred): [148.44748858]

Snapshot :

```
test data (x_test): [[1000]]
actual price (y_test): [150]
predicted price (y_pred): [148.44748858]
```

Code :

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
x = np.array([800, 1000, 1200, 1500, 1800]).reshape(-1, 1)
y = np.array([120, 150, 180, 210, 250])
x_train, x_test, y_train,y_test = train_test_split(
   x, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("test data (x_test):", x_test)
print("actual price (y_test):", y_test)
print("predicted price (y_pred):", y_pred)
print("slope (Coefficient):", model.coef_)
print("Intercept:", model.intercept_)
```

Output :

test data (x_test): [[1000]]
actual price (y_test): [150]
predicted price (y_pred): [148.44748858]

slope (Coefficient): [0.12785388]
Intercept: 20.59360730593616

Snapshot :

```
test data (x_test): [[1000]]
actual price (y_test): [150]
predicted price (y_pred): [148.44748858]
slope (Coefficient): [0.12785388]
Intercept: 20.593607305936075
```

CSE2052 - Lab Exercise 17

Program 17 : To draw the best fit regression line.

17.Design and Develop a Python program to draw the best fit regression line.

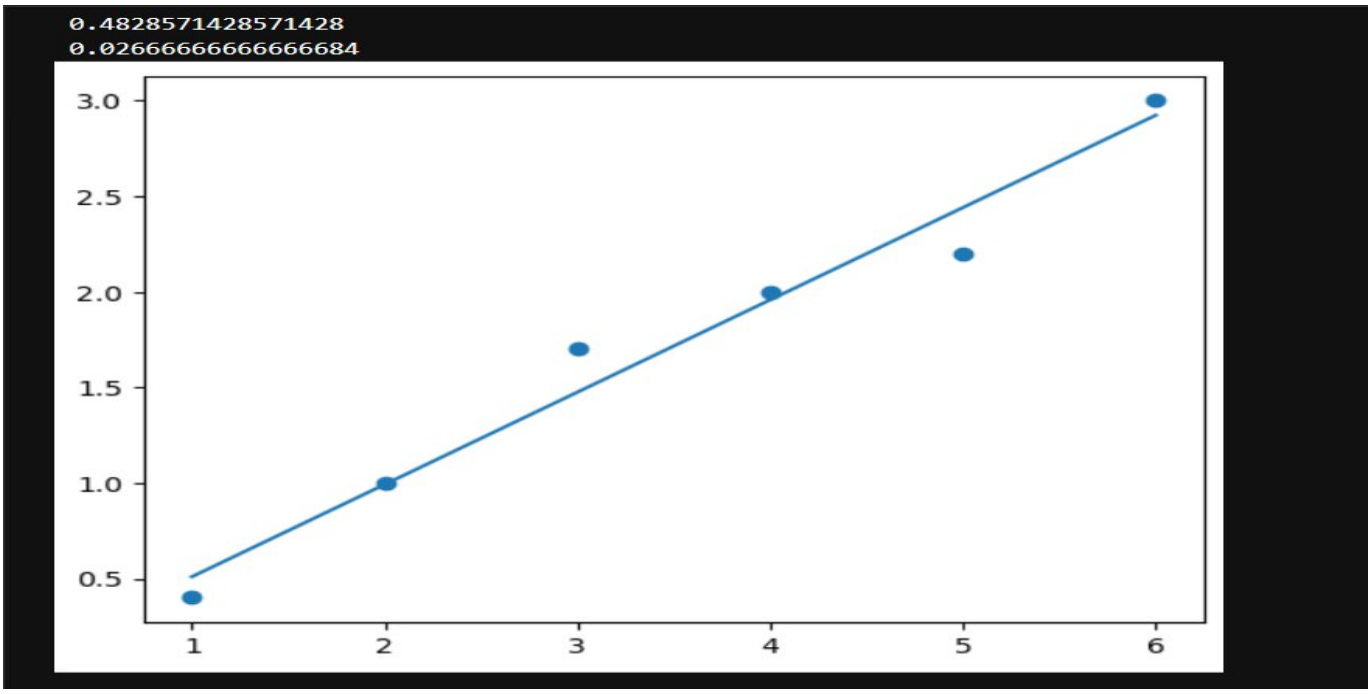
Code :

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
x=[1,2,3,4,5,6]
y=[0.4,1.0,1.7,2.0,2.2, 3.0]
slope, intercept, r, p, std_err = stats.linregress(x, y)
print(slope)
print(intercept)
def myfunc(x):
   return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```

Output :

0.4828571428571428
0.02666666666666684

Snapshot :



CSE2052 - Lab Exercise 18

Program 18 : A Decision Tree Classifier.

18. Design and Develop a python program that uses a Decision Tree Classifier to predict diabetes outcomes.

Code :

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn import metrics
from pandas import DataFrame
pima = pd.read_csv("C:/Users/Admin/Desktop/diabetes.csv")
print("Dataset Priview:")
print(pima.head)
x = list(pima.columns)
x = pima.drop("Outcome", axis=1)
y=pima["Outcome"]
print("\nPredictor Attritutes:")
print(x.columns.tolist())
x_train,x_test, y_train, y_test = train_test_split(
   x, y, test_size=0.3, random_state=1)
clf = DecisionTreeClassifier()
clf = clf.fit(x,y)
y_pred = clf.predict(x_test)
accuracy = accuracy_score(y_test, y_pred)
```

```
print("\nModel Accuracy :", accuracy)
```

Output :

```
Dataset Priview:
<bound method NDFrame.head of     Pregnancies  Glucose  BloodPressure  SkinThickness
Insulin   BMI  \
0            6      148            72            35       0  33.6
1            1       85            66            29       0  26.6
2            8      183            64             0       0  23.3
3            1       89            66            23      94  28.1
4            0      137            40            35     168  43.1
..         ...      ...           ...           ...     ...   ...
763         10      101            76            48     180  32.9
764          2      122            70            27       0  36.8
765          5      121            72            23     112  26.2
766          1      126            60             0       0  30.1
767          1       93            70            31       0  30.4

    DiabetesPedigreeFunction  Age  Outcome
0                      0.627   50        1
1                      0.351   31        0
2                      0.672   32        1
3                      0.167   21        0
4                      2.288   33        1
..                       ...  ...      ...
763                    0.171   63        0
764                    0.340   27        0
765                    0.245   30        0
766                    0.349   47        1
767                    0.315   23        0

[768 rows x 9 columns]>

Predictor Attritutes:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI',
'DiabetesPedigreeFunction', 'Age']

Model Accuracy : 1.0
```

Snapshot :

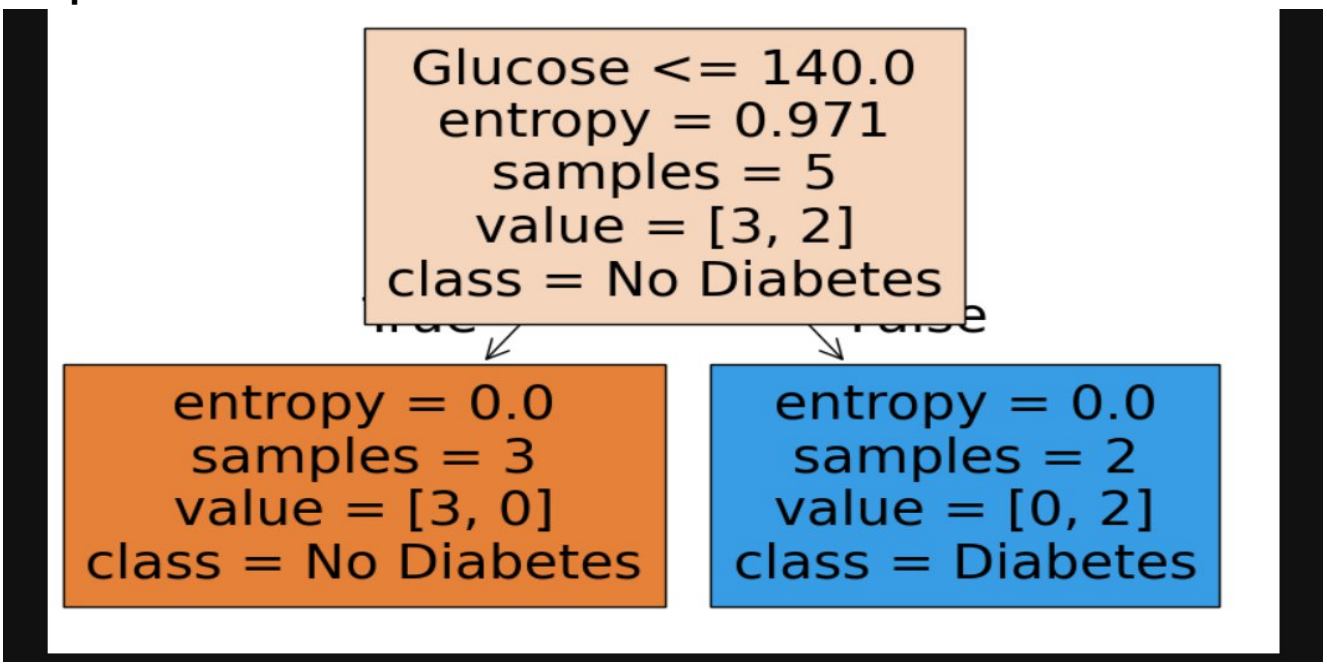
```
Dataset Priview:
<bound method NDFrame.head of      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin
BMI  \
0              6      148             72             35        0  33.6
1              1       85             66             29        0  26.6
2              8      183             64              0        0  23.3
3              1       89             66             23       94  28.1
4              0      137             40             35      168  43.1
..           ...      ...            ...            ...      ...   ...
763           10      101             76             48      180  32.9
764            2      122             70             27        0  36.8
765            5      121             72             23      112  26.2
766            1      126             60              0        0  30.1
767            1       93             70             31        0  30.4

     DiabetesPedigreeFunction  Age  Outcome
0                       0.627   50        1
1                       0.351   31        0
2                       0.672   32        1
3                       0.167   21        0
4                       2.288   33        1
..                        ...  ...      ...
763                     0.171   63        0
764                     0.340   27        0
765                     0.245   30        0
766                     0.349   47        1
767                     0.315   23        0

[768 rows x 9 columns]>

Predictor Attritutes:
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigr
eeFunction', 'Age']

Model Accuracy : 1.0
```

Code :

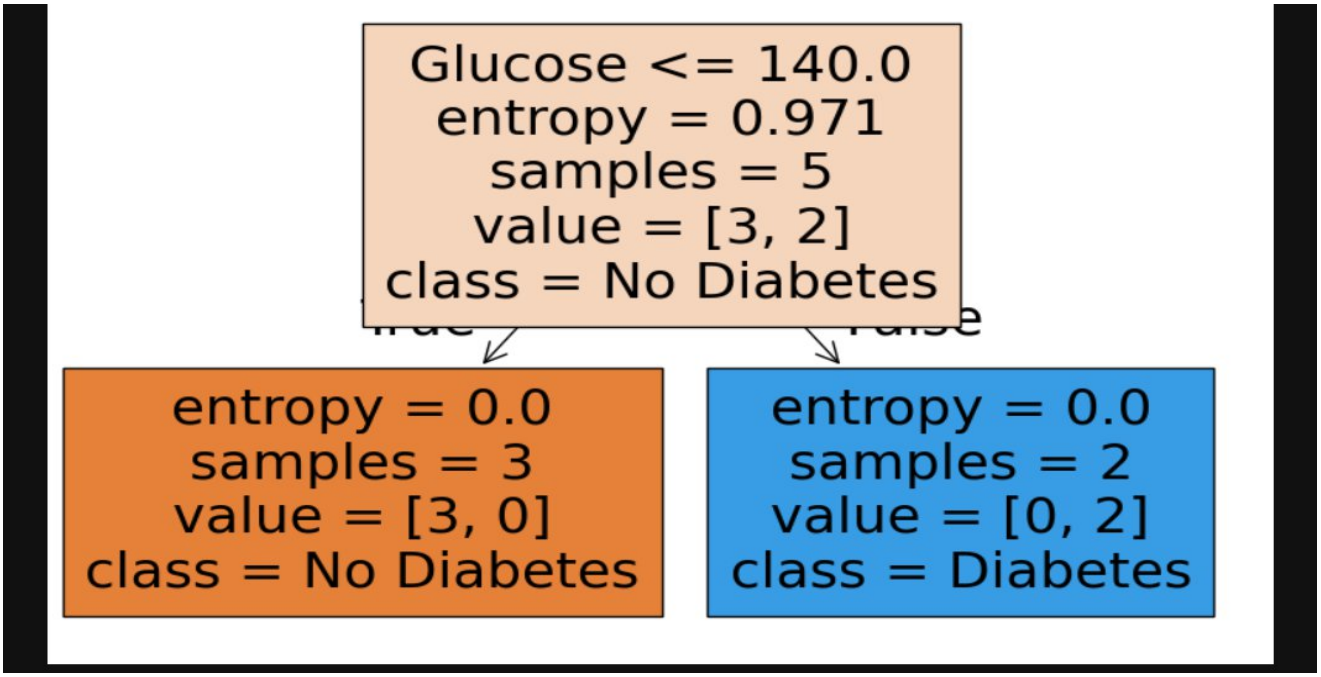
```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Data
X = [[120, 30], [85, 25], [90, 28], [160, 35], [200, 40]]
y = [0, 0, 0, 1, 1]

# Model creation and training
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

# Plot the decision tree
plt.figure(figsize=(10, 6))
plot_tree(
    model,
    feature_names=["Glucose", "BMI"],
    class_names=["No Diabetes", "Diabetes"],
    filled=True
)
```

plt.show()

Output :



Snapshot :

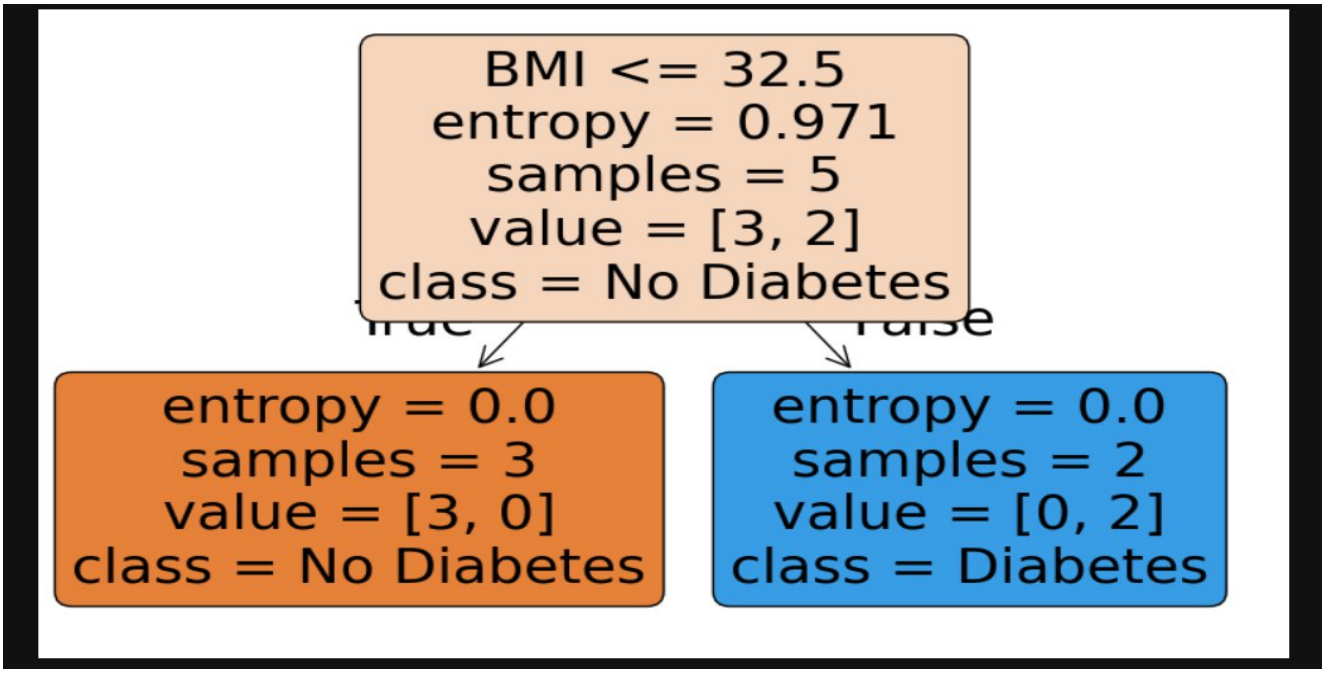


Code :

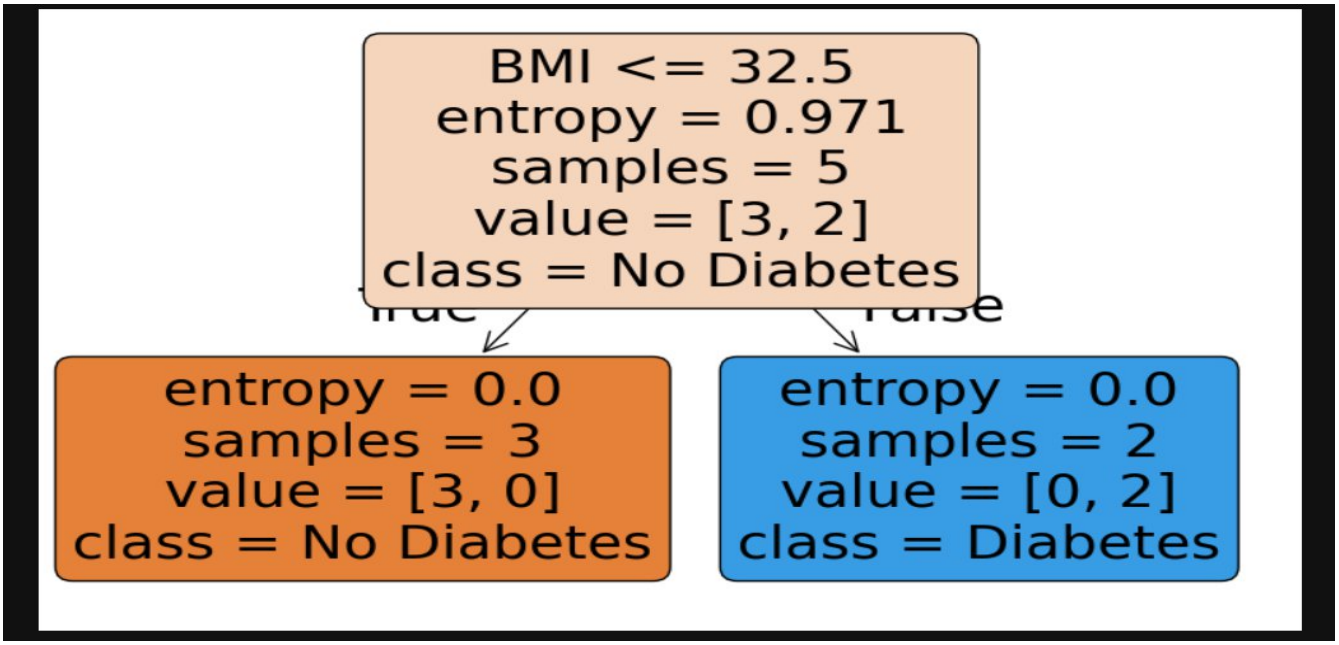
```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# Step 1: Dataset
# Features: [Glucose, BMI]
```

```
X = [
    [120, 30],
    [85, 25],
    [90, 28],
    [160, 35],
    [200, 40]
]

# Labels: 0 = No Diabetes, 1 = Diabetes
y = [0, 0, 0, 1, 1]

# Step 2: Create Decision Tree model using ENTROPY
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

# Step 3: Visualize the tree
plt.figure(figsize=(10, 6))
plot_tree(
    model,
    feature_names=["Glucose", "BMI"],
    class_names=["No Diabetes", "Diabetes"],
    filled=True,
    rounded=True
)
plt.show()
```

Output :



Snapshot :



CSE2052 - Lab Exercise 19

Program 19 : Logistic Regression

19. Write a Python program to implement Logistic Regression for a binary classification problem that predicts whether a student passes or fails based on the number of hours studies.

Code :

```
import numpy as np
from sklearn.linear_model import LogisticRegression

x = np.array([[1],[2],[3],[4],[5]])
y = np.array([0,0,0,1,1])

model=LogisticRegression()
model.fit(x, y)

hours = [[3.5]]
prediction = model.predict(hours)
probability =  model.predict_proba(hours)

print("Hours Studied : ", hours[0][0])
print("predicted class (0 = Fail , 1 = Pass) :", prediction[0])
print("probability [Fail, Pass] : ", probability[0])
```

Output :

```
Hours Studied :  3.5
predicted class (0 = Fail , 1 = Pass) : 0
```

```
probability [Fail, Pass] :  [0.5208689 0.4791311]
```

Snapshot :

```
Hours Studied :  3.5
predicted class (0 = Fail , 1 = Pass) : 0
probability [Fail, Pass] :  [0.5208689 0.4791311]
```

CSE2052 - Lab Exercise 20

Program 20 : K-means

20.Write a Python program to Implement algorithm for K-means.

Code :

```
import pandas as pd
from sklearn.cluster import KMeans

data = pd.read_csv("C:/Users/Admin/Desktop/data.csv")

model = KMeans(n_clusters=2, random_state=0)

model.fit(data)

print("Cluster labels:")
print(model.labels_)
print("\nCluster centers:")
print(model.cluster_centers_)
```

Output :

```
Cluster labels:
[1 1 1 0 0 0]

Cluster centers:
[[10.  2.]
 [ 1.  2.]]
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning:
KMeans is known to have a memory leak on Windows with MKL, when there are less chunks than
available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```

Snapshot :

```
Cluster labels:
[1 1 1 0 0 0]

Cluster centers:
[[10.  2.]
 [ 1.  2.]]
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1419: UserWarning: KMean
s is known to have a memory leak on Windows with MKL, when there are less chunks than availabl
e threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```